

UNDERSTANDING SCOPE & CLOSURES IN JAVASCRIPT



Lakshmiprasanna Vakada

What is Scope?

Scope refers to where a variable is accessible in your code.

✓ Global Scope

Variables declared outside any function or block are global and accessible anywhere.

```
let globalVar = "I am global!";

function showGlobal() {
  console.log(globalVar); // Accessible inside
function
}

showGlobal();
console.log(globalVar); // Accessible outside
function
```

◆ Global variables stay in memory and can be accessed anywhere, but overusing them can cause issues.

✓ Function Scope

Variables declared inside a function are only accessible within that function.

```
function myFunction() {  
  let localVar = "I am local!";  
  console.log(localVar);  
}  
  
myFunction();  
console.log(localVar); // ✗ Error:  
localVar is not defined
```

◆ Function-scoped variables cannot be accessed outside the function.

✓ Block Scope (let & const)

Variables declared with let or const inside {} (blocks) are limited to that block.

```
if (true) {  
    let blockVar = "I exist only in this  
block";  
    console.log(blockVar); // ✓ Works inside  
block  
}  
  
console.log(blockVar); // ✗ Error: blockVar  
is not defined
```

◆ var does NOT follow block scope, so always prefer let or const in modern JavaScript!

Closures in JavaScript

A closure is a function that remembers the variables from its parent scope, even after the parent function has executed.

```
function outerFunction() {  
    let count = 0; // Local variable  
  
    return function innerFunction() {  
        count++;  
        console.log("Count:", count);  
    };  
}  
  
const counter = outerFunction();  
counter(); // Count: 1  
counter(); // Count: 2
```

- ◆ The inner function (innerFunction) remembers count, even after outerFunction has finished execution!